

Java 22 features

Java 22 New Features with Examples

- [Unnamed Variables & Patterns \(JEP 456\)](#)
 - [Example#1: Enhanced For Loop](#)
 - [Example#2: Try-Catch Block](#)
 - [Example#3: Lambda Parameters](#)
 - [Example#4: Unnamed Patterns in a Switch Expression](#)
- [Launch Multi-File Source-Code Programs \(JEP 458\)](#)
- [Statements Before super\(...\) or this\(...\) \[Preview, JEP 447\]](#)
 - [Example: Before Java 22](#)
 - [Example: Since Java 22](#)
- [Stream Gatherers \(Preview, JEP 461\)](#)
 - [What is a Stream Gatherer?](#)
 - [Why we need Stream Gatherer?](#)
 - [What are the benefits of Stream Gatherers?](#)
 - [Example: Without Using Gatherer](#)
 - [Output:](#)
 - [Example: Using Gatherer](#)
 - [Output:](#)
- [Foreign Function & Memory API \(JEP 454\)](#)

[Example#1: Calling strlen\(\) from the C Library](#)

[Example#2: Sorting Strings Using qsort\(\)](#)

- [Class-File API \(Preview, JEP 457\)](#)

[Example:](#)

- [Scoped Values \(Second Preview, JEP 464\)](#)
- [Vector API \(Seventh Incubator, JEP 460\)](#)

Unnamed Variables & Patterns (JEP 456)

This was introduced in Java 21 as a preview feature under the name "Unnamed Patterns and Variables", and now it is finalized in Java22 as "Unnamed Variables & Patterns" without any changes.

Many a time, we define some variables or patterns that we don't actually need. In that case, we can ignore assigning them names. Instead, we can use an underscore (`_`) to denote these unnamed elements. Simply put, in situations where we declare variables that are not used later in the code, we can now replace their names with an underscore. This underscore will represent that the variable is intentionally unused. This enhancement improves code readability.

For example, Let's check how it works:

Example#1: Enhanced For Loop

```
public record Employee(String name) {}
```

```
for (Employee employee : employees) {  
    total++;  
    if (total > limit) {  
        // logic  
    }  
}
```

If we use the unnamed variable feature, we can avoid using the employee variable and replace it by the underscore.

```
for (Employee _ : employees) {  
    total++;  
    if (total > limit) {  
        //logic  
    }  
}
```

```
}
```

Example#2: Try-Catch Block

Unnamed catch blocks can be most helpful when we want to handle exceptions without needing to know the exception details

```
try {  
    int number = Integer.parseInt(string);  
} catch (NumberFormatException e) {  
    System.err.println("Not a number");  
}
```

We can transform the above code as below:

```
try {  
    int number = Integer.parseInt(string);  
} catch (NumberFormatException _) {  
    System.err.println("Not a number");  
}
```

They also work for multiple exception types in the same catch. For example:

```
catch (IllegalStateException | NumberFormatException _) { }
```

Example#3: Lambda Parameters

Suppose we want to create a simple lambda expression that prints a message. We don't actually need the parameter value, so we can use an unnamed variable:

```
// Define a lambda expression with an unnamed parameter  
public static Map<String, List<Employee>> getEmployeesByFirstLetter(List<Employee> emps) {  
    Map<String, List<Employee>> empMap = new HashMap<>();  
    emps.forEach(emp ->  
        empMap.computeIfAbsent(emp.name().substring(0, 1), _ -> new ArrayList<>()).add(emp)  
    );  
    return empMap;  
}
```

```
// Use the lambda expression with an unnamed parameter  
map.forEach( (_, _) -> System.out.println("Using unnamed Parameters"));
```

We created a lambda expression that takes an argument (denoted by `_`). The underscore (`_`) signifies that we intentionally ignore the parameter value.

Similarly, we can implement in other cases, such as Assignment Statements, Local Variable Declarations, Method References, Try-With Resources etc.

Example#4: Unnamed Patterns in a Switch Expression

Unnamed patterns allow us to simplify code by eliminating needless pattern variable names.

Suppose we have a [sealed interface](#) `Employee` with three [record](#) classes: `Salaried`, `Freelancer`, and `Intern`. We want to process each type of employee differently. Here's how we can use unnamed patterns in a switch expression:

```
sealed interface Employee permits Salaried, Freelancer, Intern { }
```

```
record Salaried(String name, long salary) implements Employee { }
```

```
record Freelancer(String name) implements Employee { }
```

```
record Intern(String name) implements Employee { }
```

```
public class EmployeeProcessor {  
    public static void main(String[] args) {  
        Employee employee = new Freelancer("Alice");  
  
        String employeeType = switch (employee) {  
            case Salaried(_, _) -> "Salaried Employee";  
            case Freelancer(_) -> "Freelancer";  
            case Intern(_) -> "Intern";  
            default -> "Unknown";  
        };  
  
        System.out.println("Employee type: " + employeeType);  
    }  
}
```

In this example, we have defined three record classes Salaried, Freelancer, and Intern that implement the Employee interface. The switch expression uses unnamed patterns as shown below:

case Salaried(_, _) matches any Salaried employee without binding the name or salary.
case Freelancer(_) matches any Freelancer employee without binding the name.
case Intern(_) matches any Intern employee without binding the name.

The default case handles other cases.

Launch Multi-File Source-Code Programs (JEP 458)

Java 11 had introduced the ability to run single-file source-code programs directly using the java launcher without compiling it explicitly. This enhancement of Java 22 allows us to run Java programs that consist of multiple files of Java source code directly, without explicitly compiling them in advance. It simplifies the transition from small programs to larger ones, which permit developers to opt when to configure a build tool necessarily. However, this approach had a limitation: All code had to be placed in a single .java file.

Let's understand it with an example. Suppose we have a directory containing two files: Employee.java and Helper.java, where each file declares a single class:

// Employee.java

```
public class Employee{  
    public static void main(String[] args) {  
        Helper.run();  
    }  
}
```

// Helper.java

```
public class Helper {  
    static void run() {  
        System.out.println("Hello!");  
    }  
}
```

With JEP 458, we can now execute this multi-file program directly using the java launcher:

\$ java Employee.java

Hello!

However, from Java 11, as soon as we add further Java files, the so-called "launch single-file source code" mechanism will no longer work. Hence, the "Launch Single-File Source Code" feature of Java 11 became a "Launch Multi-File Source Code Programs" feature in Java 22.

If we use this feature of Java 22, we can experiment without configuring build tools initially.

This detailed feature is defined in [JDK Enhancement Proposal 458](#). It also explains how this feature works in case of modules and how some exceptional cases that could theoretically occur are handled. Although, for majority of the scenarios, aforementioned description should be good enough.

Statements Before super(...) or this(...) [Preview, JEP 447]

Before Java 22 it was mandatory that the constructor of a child class had to call the constructor of the super class as the first statement to initialize the parent's fields. Java 22 has provided a flexibility on the first statement and allows other statements also before calling to the super class constructor. The same rule applies for constructor chaining also.

Simply put, we can execute code in constructors before calling super(...) or this(...).

The aim of this change is to provide relief for the rule and allow some other operations before the call to the super class constructor.

Example: Before Java 22

```
public class Animal {  
    protected String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
}  
  
public class Dog extends Animal{  
    Dog(String name, String animalName) {  
        System.out.println("call to super must be first statement in constructor"); // Error!!!  
        super(animalName);  
    }  
}
```

In the above example, it is clear that if we try to insert any statement before the super(...), the compiler will complain.

Example: Since Java 22

```
public class Animal {  
    protected String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
}  
  
public class Dog extends Animal {  
    Dog(String name, String animalName) {  
        System.out.println("I am allowed before the call to super(...)"); // Valid since Java 22  
        super(animalName);  
    }  
}
```

As shown in the example, since Java 22 it is permissible to write a statement before calling to `super(...)` constructor.

It provides more flexibility in constructor logic and can be particularly useful for preparing arguments for superclass constructors or validating constructor arguments. If we use statements before `super(...)`, we can express constructor behavior more naturally. Logic that needs to be included into helper static methods, intermediate constructors, or constructor arguments can now be placed more logically.

Stream Gatherers (Preview, JEP 461)

What is a Stream Gatherer?

A gatherer is an intermediate operation that changes a stream of input elements into a stream of output elements, with the option to take a final action once the stream of input elements has been transformed. In simple words, Stream Gatherers are custom intermediate operations that extend the capabilities of the [Java 8 Stream API](#). Technically, [Gatherer](#) is an interface in Java. In order to create a gatherer, we need to implement the Gatherer interface.

Why we need Stream Gatherer?

Existing Java Stream API has some built-in fixed set of intermediate and terminal operations for mapping, filtering, reduction, sorting, etc. Some complex tasks couldn't be expressed easily using the fixed set of operations. Stream Gatherers will help us by providing set of operations where existing intermediate operations fall short.

What are the benefits of Stream Gatherers?

Gatherers provide more flexibility and expressiveness for stream pipelines. They allow us to manipulate data streams in ways that were previously complex or not directly supported by the existing built-in intermediate operations. They offer more customized stream operations, enhancing code reusability for stream-based tasks. Gatherers simplify the understanding and implementation of complicated stream operations.

Example: Without Using Gatherer

Let's take an example of a complex task that groups elements into fixed-size groups of three, but retain only the first two groups. Therefore, the stream `[0, 1, 2, 3, 4, 5, 6, ...]` should produce `[[0, 1, 2], [3, 4, 5]]`. Since there is no intermediate operation for this scenario, we will accomplish it by writing below method:

```
public static ArrayList<ArrayList<Integer>> findGroupsOfThree(long fixed_size, int grouping) {

    return Stream.iterate(0, i -> i + 1)

        .limit(fixed_size * grouping)

        .collect(Collector.of(

            () -> new ArrayList<ArrayList<Integer>>(),

            (groups, element) -> {

                if(groups.isEmpty() || groups.getLast().size() == fixed_size) {

                    var current = new ArrayList<Integer>();

                    current.add(element);

                    groups.addLast(current);

                }

                else {

                    groups.getLast().add(element);

                }

            },

            (left, right) -> {

                throw new UnsupportedOperationException("Parallelization can't be done");

            }

        ));

}
```

Output:

```
[[0, 1, 2], [3, 4, 5]]
```

Example: Using Gatherer

Let's consider using `gather()` operation of the Gatherer API to simplify the `findGroupsOfThree()` method:

```
public static List<List<Integer>> findGroupsOfThreeWithGatherer(long fixed_size, int grouping) {

    return Stream.iterate(0, i -> i + 1)

        .gather(Gatherers.windowFixed((int)fixed_size))

        .limit(grouping)

        .collect(Collectors.toList());
}
```

Output:

```
[[0, 1, 2], [3, 4, 5]]
```

As shown in the example above, we have used a new method **`gather()`** of the Stream API in JDK 22. The Stream Gatherers API defines the `Stream.gather(...)` method and a Gatherer interface.

The **`windowFixed()`** method returns a Gatherer that gathers elements into windows, i.e. we can group stream elements in lists of predefined sizes.

The output upon successful execution of the `findGroupsOfThreeWithGatherer(long, int)` method is the same: `[[0, 1, 2], [3, 4, 5]]`. Undoubtedly, this is much easier to read and maintain.

In order to get more detail on Stream Gatherers, kindly visit [Gatherer Documentation](#).

Foreign Function & Memory API (JEP 454)

After a total of eight incubator and three preview versions, the Foreign Function & Memory API in Java 22 is finally being finalized by [JDK Enhancement Proposal 454](#). This enhancement enables Java programs to interoperate with code and data outside of the Java runtime.

The Foreign Function & Memory API (FFM API) makes it possible to access foreign functions (code outside the JVM) and foreign memory (memory not managed by the JVM in the heap) from Java. It connects the gap between Java and native code (e.g., C/C++ libraries). The FFM API is proposed to replace the highly complicated, error-prone, and slow Java Native Interface (JNI). Here are some of the examples:

Example#1: Calling `strlen()` from the C Library

Suppose we want to call the `strlen()` function from the standard C library. Here's how we can do it:

```
import jdk.incubator.foreign.CLinker;

import jdk.incubator.foreign.MemoryAddress;

public class StringLengthExample {

    public static void main(String[] args) {

        String text = "Hello, world!";

        MemoryAddress address = CLinker.toCString(text);

        long length = CLinker.systemDefault().lookup("strlen").invokeExact(address);

        System.out.println("Length of the string: " + length);

    }

}
```

Example#2: Sorting Strings Using `qsort()`

Let's sort an array of strings using the `qsort()` function from the C library. We'll provide a Java callback function to compare elements:

```
import jdk.incubator.foreign.CLinker;
import jdk.incubator.foreign.MemorySegment;
import java.util.Arrays;

public class StringSortExample {
    public static void main(String[] args) {
        String[] strings = {"Java", "Angular", "JavaScript", "Scala"};
        MemorySegment segment = CLinker.toCStringArray(strings);
        CLinker.systemDefault().lookup("qsort").invokeExact(segment, strings.length,
            CLinker.systemDefault().addressOf(String::compareTo));
        Arrays.stream(strings).forEach(System.out::println);
    }
}
```

Class-File API (Preview, JEP 457)

Sometimes it becomes necessary to examine and extend programs without altering source code. This enhancement offers a standard API for generating, parsing, and transforming Java class files. It focuses on working with class files (bytecode) in an easier manner.

The Class-File API allows developers to process class files according to the Java Virtual Machine Specification. It bridges the gap between Java and native code by providing a consistent way to work with class files. The objective is to replace the JDK's internal copy of third-party libraries like ASM in the long run.

Class files serve as the common language for the Java ecosystem. Existing class-file libraries (e.g., ASM, BCEL, Javassist) have different design goals and progress at different rates. The JDK's internal class-file library needs to keep pace with the changeable class-file format.

Example:

Let's observe how to read and analyze a class file using the Class-File API:

```
import java.io.IOException;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.List;
import java.util.stream.Collectors;
import jdk.incubator.foreign.ClassFile;
import jdk.incubator.foreign.ClassFileParser;

public class ClassFileExample {
    public static void main(String[] args) throws IOException {
        Path classFilePath = Paths.get("MyClass.class");
        ClassFile classFile = ClassFileParser.parse(classFilePath);

        // Extract class name and interfaces
        String className = classFile.thisClass().name();
        List<String> interfaces = classFile.interfaces().stream()
            .map(ClassFile.ConstPoolEntry::name)
            .collect(Collectors.toList());
    }
}
```

```
System.out.println("Class name: " + className);

System.out.println("Interfaces: " + interfaces);

}

}
```

In this example, we parsed a class file (MyClass.class) using the Class-File API. We extracted the class name and interfaces defined in the class file.

In order to try the examples in JDK 22, we must enable preview features as follows:

Compile the program with `javac --release 22 --enable-preview Main.java`.

Run it with `java --enable-preview Main`.

In summary, the Class-File API simplifies class-file processing, ensures compatibility with evolving class-file formats, and provides a consistent way to work with bytecode

Scoped Values (Second Preview, JEP 464)

Scoped values were introduced in Java 21 as a preview feature. Scoped values again considered as a second preview in Java 22 without any changes.

Scoped values allow passing one or more values to one or more methods without explicitly defining them as parameters. They are an alternative to thread-local variables, especially when dealing with a large number of virtual threads. Scoped values are preferred for their safety and efficiency.

Vector API (Seventh Incubator, JEP 460)

This enhancement aims to provide an API for expressing vector computations that compile at runtime to optimal vector instructions on supported CPU architectures. Essentially, it allows developers to work with vectorized operations efficiently. The Vector API enables expressing vector computations using a concise and platform-agnostic API. The API is designed for performance and portability.